

Como Aplicar a Migration (Criar Tabela PostgreSQL)

MÉTODO MAIS FÁCIL — Script Automático

Criamos um script Python que faz **TUDO automaticamente** para você!

Passo 1: Pegar a DATABASE_URL do Railway

1. Acesse [Railway](#)
2. Entre no projeto **BOT-SNIPER**
3. Clique em **PostgreSQL** (o addon do banco)
4. Vá em **Variables** (ou **Connect**)
5. Copie o valor de **DATABASE_URL** (começa com `postgresql://...`)

Passo 2: Rodar o script

Opção A — No seu computador local:

```
# 1. Cole a DATABASE_URL que você copiou:
export DATABASE_URL="postgresql://postgres:senha123@região.railway.app:5432/railway"

# 2. Rode o script:
python3 aplicar_migration.py
```

Opção B — Via Railway CLI (se tiver instalado):

```
# Se você tiver Railway CLI instalado:
railway run python3 aplicar_migration.py
```

Passo 3: Verificar se funcionou

Se tudo der certo, você vai ver:

```
✓ MIGRATION CONCLUÍDA COM SUCESSO!
✓ Tabela criada com 8 colunas:
  - id: bigint
  - criado_em: timestamp with time zone
  - data: date
  - hora: time without time zone
  - evento: character varying
  - moeda: character varying
  - timeframe: character varying
  - payload: jsonb
📊 Registros na tabela: 0
```

Pronto! ✓ A partir de agora, **TODOS os eventos serão salvos no PostgreSQL** e vão sobreviver aos redeploys!

Alternativa: Fazer Manualmente (se o script não funcionar)

Via Railway Web Interface

1. Acesse [Railway](#) → seu projeto
2. Clique em **PostgreSQL** → **Data** → **Query**
3. Cole este SQL e clique em **Run**:

```
CREATE TABLE IF NOT EXISTS public.crypto_eventos (
  id BIGSERIAL PRIMARY KEY,
  criado_em TIMESTAMP WITH TIME ZONE DEFAULT NOW() NOT NULL,
  data DATE NOT NULL,
  hora TIME NOT NULL,
  evento VARCHAR(50) NOT NULL,
  moeda VARCHAR(20),
  timeframe VARCHAR(10),
  payload JSONB NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_crypto_eventos_data ON public.crypto_eventos(data);
CREATE INDEX IF NOT EXISTS idx_crypto_eventos_moeda ON public.crypto_eventos(moeda);
CREATE INDEX IF NOT EXISTS idx_crypto_eventos_evento ON public.crypto_eventos(evento);
CREATE INDEX IF NOT EXISTS idx_crypto_eventos_moeda_data ON public.crypto_eventos(moeda, data);

SELECT 'Tabela criada com sucesso!' AS status;
```

1. Se aparecer `Tabela criada com sucesso!` → funcionou! 

Depois da Migration

O que muda automaticamente:

1. **Bot salva no PostgreSQL:** Todo evento (ENTRADA/SAIDA/BLOQUEADO) vai para o banco
2. **Dados sobrevivem:** Redeploys não apagam mais nada
3. **Resumo diário funciona:** `/resumo` vai mostrar dados permanentes
4. **Telegram 23:59 UTC:** Relatório vai ter histórico completo

Como testar:

```
# Ver resumo no navegador:
curl https://web-production-77454.up.railway.app/resumo?formato=texto

# Forçar envio ao Telegram agora:
curl https://web-production-77454.up.railway.app/resumo?enviar=1
```

Problemas?

“DATABASE_URL não encontrada”

- Certifique-se de copiar a URL completa do Railway
- A URL deve começar com `postgresql://`

“Nenhum driver PostgreSQL instalado”

```
pip3 install psycopg2-binary
# ou
pip3 install pg8000
```

“Tabela já existe”

- **Não é erro!** Significa que a migration já foi aplicada antes
- Pode ignorar tranquilamente 

Verificar depois

Após aplicar a migration, você pode rodar este SQL no Railway para conferir:

```
-- Ver estrutura da tabela
\d crypto_eventos

-- Contar eventos salvos
SELECT COUNT(*) FROM crypto_eventos;

-- Ver últimos 10 eventos
SELECT criado_em, evento, moeda, timeframe
FROM crypto_eventos
ORDER BY id DESC
LIMIT 10;
```

Resumo: Execute `aplicar_migration.py` com a DATABASE_URL do Railway e pronto! 